装軌車両の不整地走行を模倣学習（ACT）で再現する
— 突堤の通過（途中経過レポート）—

Physical AI 応用編1 最終課題　／　Ignition Gazebo ＋ ROS 2 Humble ＋ LeRobot（ACT）

1. 概要と目的

建設機械・災害対応ロボット・惑星探査ローバーなどの不整地走行ロボットでは、地形に応じて速度を切り替える操縦技能が必要になる。本課題では、Ignition Gazebo 上に砲塔付きの**装軌車両**（**多輪スキッドステアで履帯を模擬**）を自作し、操縦経験のある人のテレオペから **ACT**（**Action Chunking with Transformers**）を学習して、速度の切り替えを自律的に行わせる。

本レポートは第1の地形「**突堤**」（**台形の山**）の検証結果である。突堤を全開のまま越えると車体が跳ねて危険だが、ゆっくり走り続けると時間がかかる。「**登りは速く、頂上の手前で減速し、越えたら再び加速する**」という使い分けを、10 回のデモから学習できるかを確かめた。第2の地形（砲身が壁に当たる対戦車壕）は今後の課題である。

結果の要約

学習した方策（ACT）は、10 試行すべてで**跳ねずに通過**した。所要時間は全開のルールベースと同等以上（3.1 秒対 3.8 秒）で、跳ねないルールベース（一定の低速）の 2〜4 倍速かった。一方で、行動チャンクの実行ステップ数を学習時の設定（1）のままにすると跳ねずに通過できたのは 0/10 にとどまり、この設定が結果を大きく左右することも分かった（第5章）。

2. システム構成

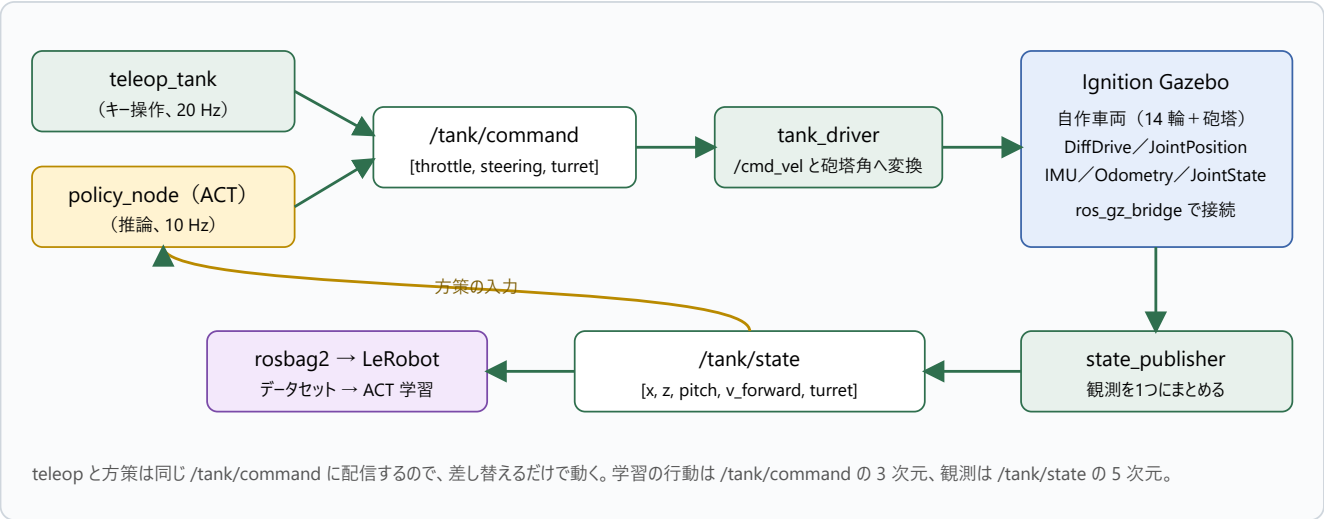


図1　システム構成。車両・地形・操縦系・観測系はすべて自作。

車両モデル（自作・架空のパラメータ）

- 車体 2.0×1.0×0.4 m、総質量約 445 kg。片側 7 輪（半径 0.15 m）を詰めて配置し、DiffDrive に左右 14 輪の関節をすべて登録して**スキッドステア**とした。横方向の摩擦を小さくして、その場旋回できるようにした。
- 履帯は見た目用の箱で、衝突判定は車輪が担当する。砲塔は鉛直軸の関節（JointPositionController）で回し、**砲身にも衝突判定を付けた**（第2の地形で壁に当たる状況を再現するため）。
- 慣性は形状と質量から計算して生成する（`tools/gen_tank_model.py`）。

操縦系・観測

- **行動** `[throttle, steering, turret_angle]`：アクセル（−1〜1、0.25 刻み）、ハンドル（左右、中立キーで戻す）、砲塔角。操縦経験者の意見でキー割り当てを決めた。全開は 4 m/s。
- **観測** `[x, z, pitch, v_forward, turret_angle]`：世界座標の位置（OdometryPublisher の真値）、IMU のピッチ角（機首上げを正）、車輪オドメトリの速度、砲塔角。地形の局面（登り・頂上・下り）を見分けられるようにした。

地形：突堤

高さ 1.0 m、斜面 35°、頂上の平らな部分 0.3 m の台形。頂上の縁を鋭くして、全開で越えると車体が跳ねるようにした（`tools/gen_berm.py` で生成）。当初は斜面 25°・頂上 2.0 m だったが、操縦者の「もっと尖らせて、急にして」という意見で変更した。

3. 学習の方法

- **デモ**：操縦経験者のテレオペ 10 エピソード（突堤、毎回同じ初期位置）。登りは約 3 m/s、頂上で 0〜0.25 まで減速、下りで段階的に加速。ハンドルと砲塔は操作しない。
- **変換**：`/tank/state` と `/tank/command` を 10 Hz に再サンプルして LeRobotDataset 化（815 フレーム）。操縦の前後の停止区間は切り詰めた（第5章）。
- **学習**：ACT（chunk 16 ステップ＝1.6 秒、バッチ 8、学習率 1e-4、5000 ステップ）。GPU なしの CPU で約 23 分。loss は 89 から約 0.14 に低下した（図2）。
- **推論**：`/tank/state` を受けて 10 Hz で `/tank/command` を配信。行動チャンクを 8 ステップ続けて実行する。制御周期は、シミュレーション時刻で 13.3 Hz（デモの時間の刻みに合わせた。第5章）。

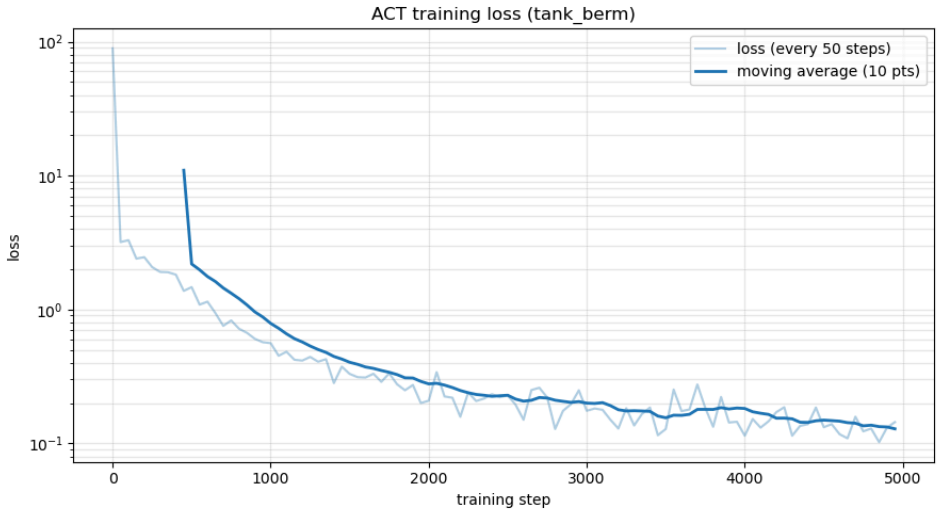


図2　学習 loss の推移（縦軸は対数）。

4. 結果

評価は、車両を初期位置に戻して走行させ、x ≧ 13 m（山を越えた先）に達するまでを 1 試行とし、各条件 10 試行ずつ行った。「**跳ねずに通過**」は、通過し、かつ最高の高さ z < 1.70 m、かつ空中時間 < 0.6 秒（IMU の加速度の大きさが 3 m/s² 未満の時間）とした。閾値は、操縦者のデモ（最高 z が 1.40〜1.61 m）と全開（約 1.84 m）の間に置いた。所要時間はシミュレーション時刻で測った。

方式	通過	跳ねずに通過	最高 z（平均）	空中（平均）	所要時間（平均）
ルールベース：全開（1.0）一定	10/10	0/10	1.84 m	0.87 s	3.8 s
ルールベース：0.5 一定	10/10	10/10	1.57 m	0.50 s	7.2 s
ルールベース：0.25 一定	10/10	10/10	1.46 m	0.34 s	13.7 s
ACT（行動チャンクの実行 1 ステップ）	10/10	0/10	1.71 m	0.88 s	3.0 s
ACT（実行 8 ステップ）	**10/10**	**10/10**	**1.44 m**	**0.26 s**	**3.1 s**
ACT（実行 16 ステップ）	10/10	7/10	1.53 m	0.42 s	4.1 s

速く走ると跳ね、跳ねないようにすると遅いというトレードオフが、ルールベースの結果にはっきり出ている。ACT（8 ステップ）は、全開と同じ程度の速さのまま、頂上の手前で減速して跳ねを避けた。これは、状況（ピッチと位置）に応じた速度の切り替えを方策が獲得したことを示す。

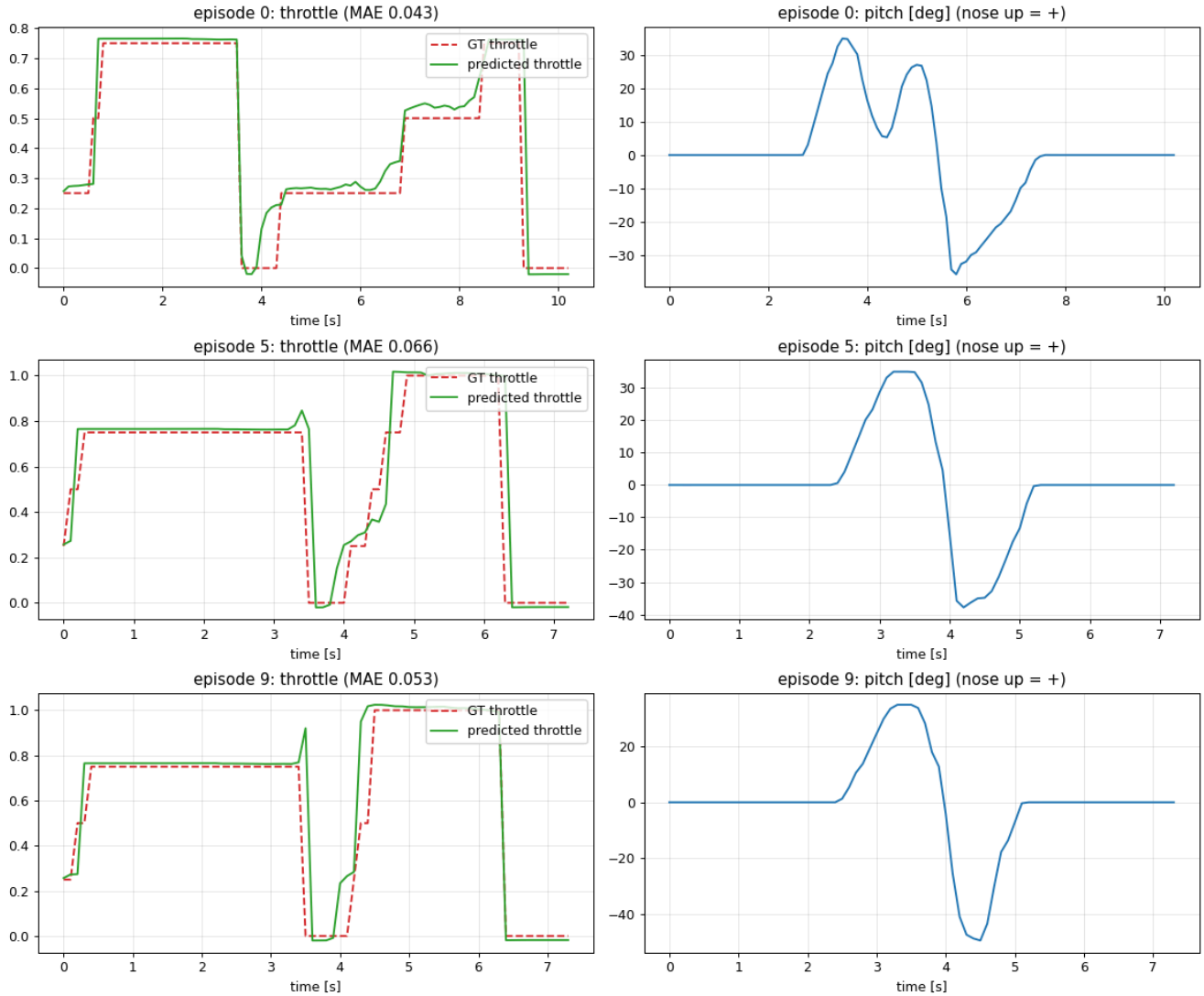


図3　学習データ上の予測（緑）と操縦者の行動（赤の破線）。平均絶対誤差 0.04〜0.07。右はピッチ。

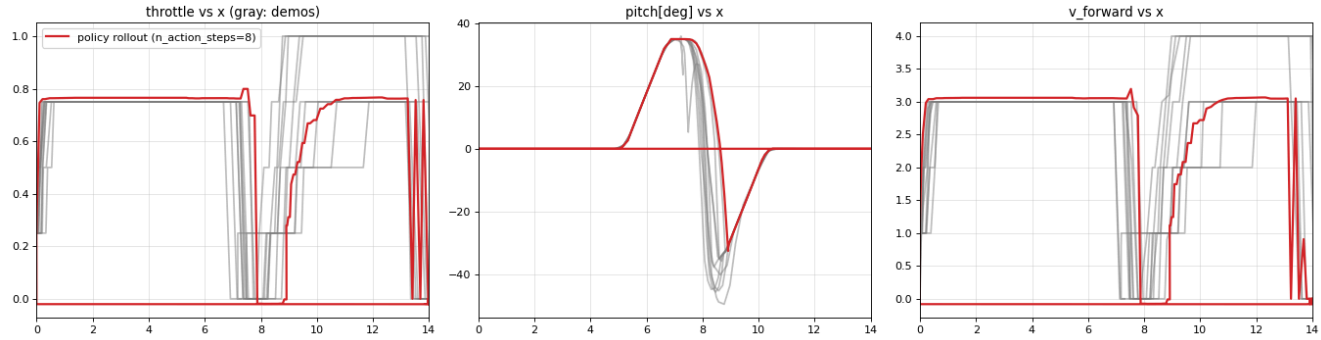


図4　方策の走行（赤）と操縦者のデモ（灰色）。左：スロットル、中：ピッチ、右：速度（横軸は車両の x）。頂上（x≈7.5 m）で減速し、越えてから加速している。

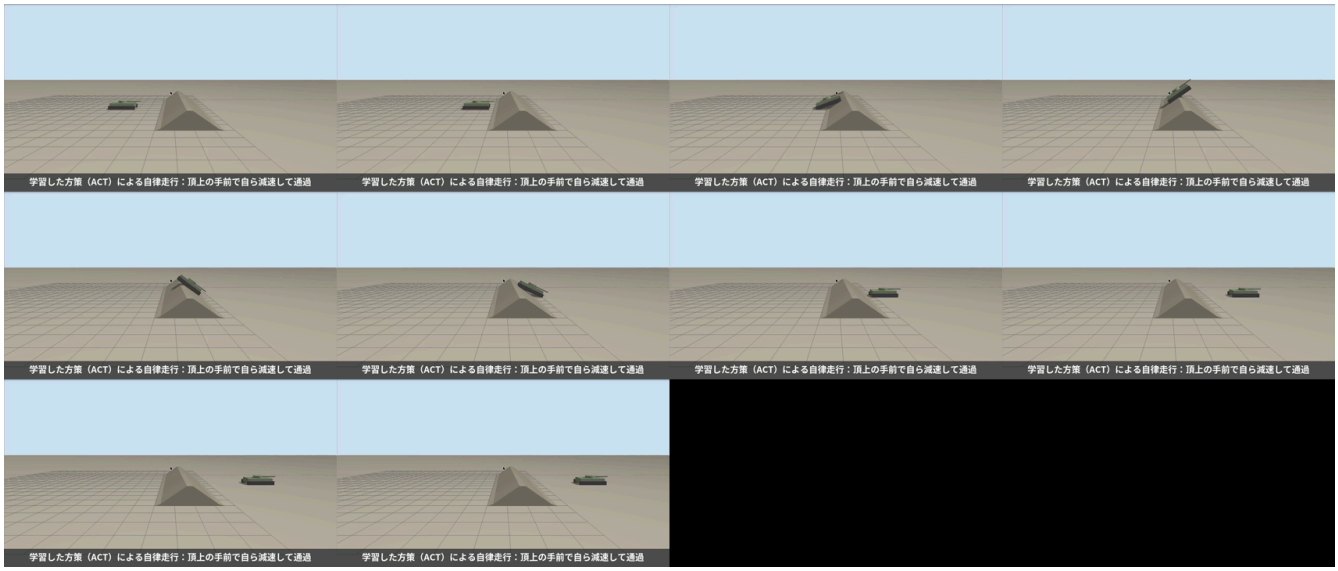


図5　方策の走行の動画から切り出したコマ（左上から右下へ）。頂上でも跳ね上がらずに越えている。

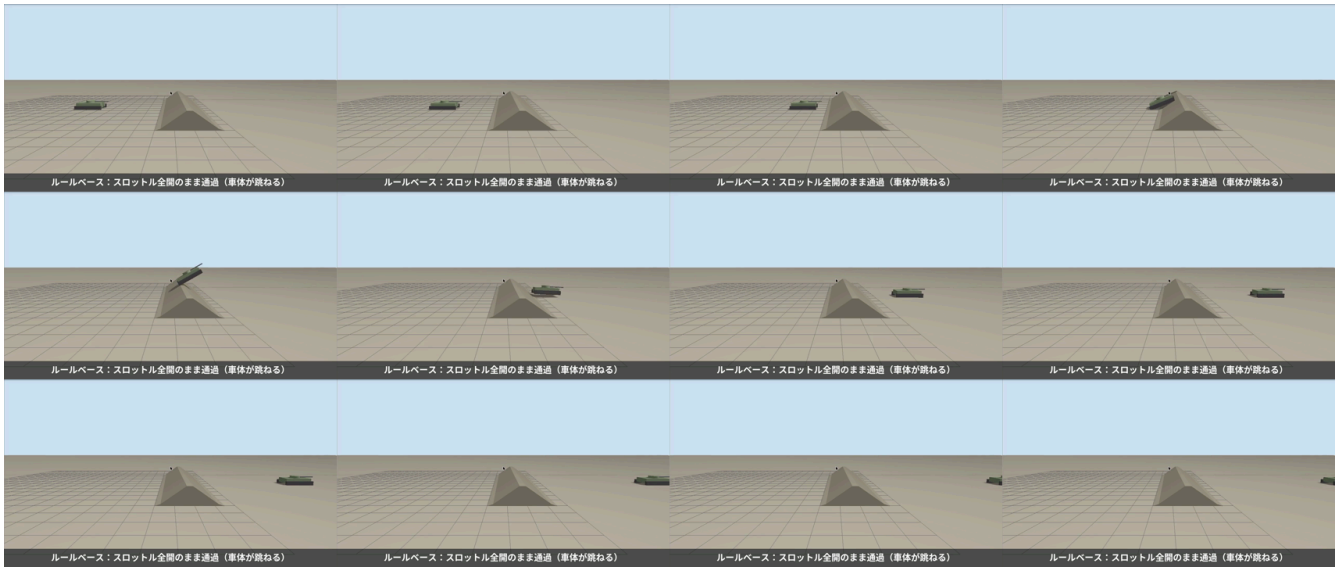


図6　比較：全開のままの通過。頂上で車体が大きく跳ね、機首を下げて着地している。

5. 失敗と考察

失敗1：方策が動き出せない（開始前の停止区間）

最初に学習した方策は、学習データ上の予測は良好（平均絶対誤差 0.09〜0.12）だったのに、実行すると動き出さなかった。各エピソードの先頭には、操縦を始める前の停止区間が 1.5〜3.6 秒あり、その間の観測は毎回同じ（x=0、速度 0）なのに、動き出す時刻は毎回違う。方策には時刻の情報がなく、最頻値である「止まったまま」を出力していた。実行では、後ろへ 0.8 m ずるずる下がり、観測が変わってからようやく動き出した。**対策**として、変換時に、操縦を始める前と終えた後の停止区間を切り詰めた（1289 → 815 フレーム）。予測の誤差は約半分になり、最初のフレームから動き出せるようになった。

失敗2：頂上で減速しない（行動チャンクの実行ステップ数）

次に、実行したところ、通過はするが**頂上でスロットルを 0.79 から 1.0 へ上げてしまい**、跳ねずに通過できたのは 0/10 だった（図7）。デモでは、頂上で減速する位置が x=6.8〜7.8 m と約 1 m ばらついていたため、頂上付近のほぼ同じ状態に対して「落とす」と「保つ」の 2 つの答えがあり、方策が決めきれなかった。さらに、毎ステップ行動チャンク（1.6 秒先まで）の先頭 1 個だけを実行していたため、「少し先で減速する」という計画の手前の部分しか使われていなかった。いったん減速が遅れるとデモに無い状態に入り、さらに外れる（共変量シフト）。

対策として、1 回の推論で得た行動チャンクを 8 ステップ（0.8 秒）続けて実行したところ、10/10 で跳ねずに通過した。再学習は不要だった。ただし長すぎる 16 ステップでは 7/10 に下がり、状況の変化に追従できなかった。**ACT の行動チャンクの考え方が、このタスクで効くこと**が確認できた。

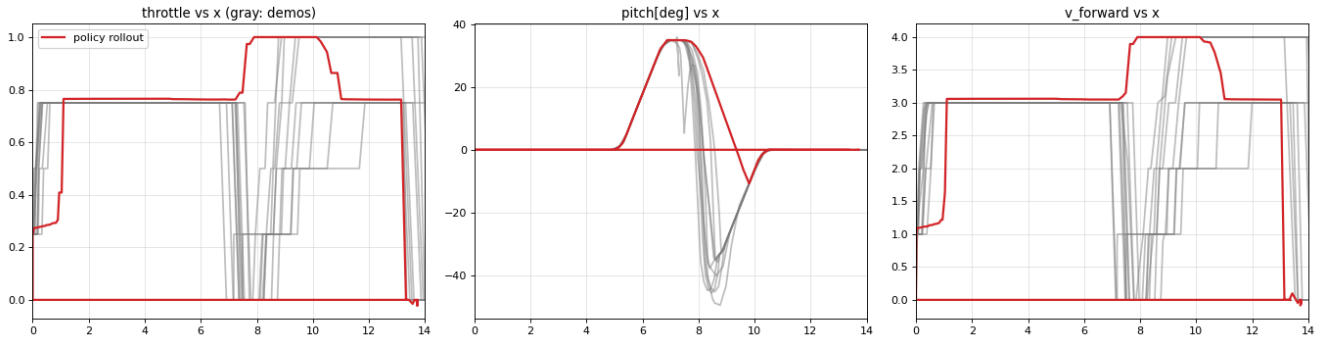


図7　失敗例（実行 1 ステップ）：頂上（x≈7.4 m、ピッチ 35°）でスロットルを上げてしまい、速度が 4 m/s になって跳んだ。灰色はデモ。

失敗3：評価結果が実行のたびにばらつく（時間の基準のずれ）

同じ方策・同じモデルなのに、「跳ねずに通過」の回数が 4/10〜10/10 と実行のたびに変わった。原因は**時間の基準**だった。方策は**実時間**の 10 Hz で指令を出していたが、シミュレーションは実時間の 0.8〜0.98 倍の速さ（RTF）で揺れて動く。さらに、デモは GUI 付き（RTF 約 0.75）で記録し、rosbag は**実時間**でタイムスタンプを付けるため、データの「0.1 秒」はシミュレーション時間では約 0.075 秒に当たる。制御周期をシミュレーション時刻で測り、周期を変えて評価したところ、**13.3 Hz（= 10 ÷ 0.75）で 10/10**、10 Hz で 7/10、8 Hz で 1/10 となった（実時間の 10 Hz は 4〜10/10 でばらついた）。**対策**として、方策の制御周期をシミュレーション時刻の 13.3 Hz に固定し、評価ノードも所要時間をシミュレーション時刻で測るようにした。今後のデモは `ros2 bag record --use-sim-time` でシミュレーション時刻のタイムスタンプで記録する（このとき制御周期は 10 Hz でよい）。

その他の工夫・遭遇した問題

- **真値の位置姿勢**：PosePublisher はリンクの相対姿勢しか出さず、SceneBroadcaster の姿勢は bridge で名前が失われた。OdometryPublisher（3 次元）で世界座標の真値を取った。
- **評価の公平性**：最初のルールベースの測定は、学習と同時に走らせたためシミュレーションが遅れ、所要時間が長く出た。シミュレーション時刻で測るよう直して測り直した。

- **共有環境での干渉**：ユーザーの画面上のシミュレーションと試験用のシミュレーションが同じ ROS トピックを共有し、互いに指令が混ざった。試験用は ROS ドメインと Gazebo パーティションを分けて隔離した。
- **評価指標**：着地の最大加速度は同じ条件でも 26〜662 m/s² とばらつき（接触計算のスパイク）、指標には空中時間と最高 z を使った。
- **安全**：`tank_driver` に 0.5 秒のウォッチドッグを入れ、指令が途絶えたら停止する。

限界と今後

- シミュレーションはほぼ決定的で、毎回同じ初期位置から始めるため、10 試行は独立ではない。初期位置や地形の寸法を変えた汎化の評価は今後の課題である。
- 行動チャンクの実行ステップ数は評価結果を見て選んでおり、評価と調整に同じ環境を使っている。
- 次の課題は、**対戦車壕**（砲塔が前向きだと砲身が向こう側の壁に当たり、90° 回すと当たらない）で、砲塔の操作を含めた学習・評価を行うことである。

6. 再現手順と参考

コード一式（モデル・地形の生成スクリプト、ROS 2 パッケージ、変換・学習の設定、評価スクリプト、README の手順）は GitHub のリポジトリにある。評価結果の全試行の記録は `results/` にある。

参考：Qiita「gazeboとROS2を連携してロボットを操作する」（N622）— ROS 2 と Gazebo の連携の考え方を参考にした（コードとモデルは自作）。matsuolab/pai_training — rosbag から LeRobotDataset への変換と ACT の学習・推論の仕組みを参考にした。LeRobot（Hugging Face）。T. Z. Zhao et al., “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware”, 2023（ACT）。車両のパラメータはすべて架空の値である。